

SISTEMA DE LICENCIAS

Reglas de Negocio, Arquitectura y Diseño

Versión 1.0 · Abril 2026

Documento de diseño — No implementado aún

1. Visión General del Sistema

El sistema permite gestionar licencias de múltiples servicios de e-commerce (páginas hijas) desde una plataforma centralizada (página madre). Cada página hija verifica periódicamente su licencia contra la madre y aplica bloqueos automáticos si el cliente no está al día.

1.1 Arquitectura General

Componentes principales

- Página Madre (Next.js): fuente de verdad, gestión de clientes, pagos y licencias.
- Páginas Hijas (Node.js + EJS): e-commerces suscritos que consumen la licencia.
- Módulo license-guard: paquete JS reutilizable que se instala en cada hija.
- Endpoint /api/license/verify: único punto de comunicación entre hija y madre.

1.2 Flujo de Verificación (Pull-based)

Las hijas no reciben notificaciones de la madre (push). Ellas consultan activamente cuando corresponde. No se usan cronjobs por limitaciones de los planes gratuitos de Render/Firebase.

Trigger de verificación

La verificación ocurre ÚNICAMENTE cuando el administrador de la hija hace login.
Si no pasaron N días desde la última consulta → usa estado cacheado local.
Si ya pasaron N días → consulta al endpoint de la madre.

2. Arquitectura de Bloqueo

El bloqueo opera en dos capas independientes. El admin puede intentar saltarse el panel, pero el cliente final siempre queda bloqueado porque el middleware actúa a nivel de servidor.

2.1 Capa Backend (Middleware Express)

Comportamiento del middleware

Intercepta TODAS las rutas antes de que lleguen al cliente final.

Si estado = suspended → devuelve página genérica "Servicio no disponible".

El cliente nunca accede al e-commerce. No depende de ninguna acción del admin.

No se puede saltar inspeccionando el código frontend.

2.2 Capa Admin (Panel de Administración)

Comportamiento del panel admin

Muestra banner o pantalla completa indicando "Licencia vencida".

Incluye enlace directo a la página de pago en la madre.

El admin VE el aviso pero no puede operar normalmente.

Aunque el admin logre saltar el panel, el e-commerce ya está bloqueado por el middleware.

3. Tipos de Plan

Tipo	Ciclo	Frecuencia Verificación	Notas
monthly	Mensual (billing_day)	Cada 3 días	Ciclo corto, detección rápida de impago
annual	Anual (billing_day)	Cada 7 días	Más margen, mismo mecanismo
lifetime	Sin vencimiento	Cada 30 días	Solo para clientes muy selectos

3.1 Plan Vitalicio (Lifetime) — Caso Especial

Reglas del plan vitalicio

- No tiene expires_at ni billing_day.
- No genera pagos recurrentes.
- Estado siempre active a menos que sea revocado manualmente.
- La hija verifica cada 30 días para detectar revocaciones manuales.
- Revocación: se registra cancelled_at manualmente desde el panel de la madre.

4. Estados de Licencia

El estado NO se almacena como campo fijo en la DB. Se calcula en tiempo real cada vez que se consulta, comparando la fecha actual contra `expires_at` y `grace_days`.

Estado	Condición	Efecto
active	<code>today <= expires_at</code>	Todo funciona normal
grace_period	<code>expires_at < today <= expires_at + grace_days</code>	Funciona con aviso en panel admin
suspended	<code>today > expires_at + grace_days</code>	Bloqueo total: middleware + panel
cancelled	<code>cancelled_at != null</code>	Por encima de todo. Bloqueo inmediato.
trial	Período de prueba configurado	Opcional, mismo mecanismo que active

4.1 Prioridad de Estados

Orden de evaluación

1. `cancelled_at != null` → cancelled (por encima de todo)
2. `planType = lifetime` y no revocado → active
3. `today <= expires_at` → active
4. `expires_at < today <= expires_at + grace_days` → grace_period
5. `today > expires_at + grace_days` → suspended

4.2 Comportamiento al Cancelar

Cuando se registra `cancelled_at`, el servicio NO corta inmediato si ya había un período pagado. Respeta el `expires_at` actual y recién bloquea cuando esa fecha vence.

Ejemplo de cancelación

Cliente cancela el 10 de abril.

Su `expires_at` era 7 de mayo.

- Sigue activo hasta el 7 de mayo.
- El 8 de mayo entra en `grace_period`.
- Al agotar la gracia → suspended definitivo.

5. Período de Gracia

5.1 Gracia del Cliente (Días Post-Vencimiento)

Días extra que se le dan al cliente después de que vence su licencia antes de suspenderlo. Configurado por licencia (`grace_days`). Mentalidad Netflix: esos días no son gratis, se cobran en el siguiente pago.

5.2 Gracia del Endpoint (Fallo de Conexión)

Días que la hija tolera sin poder contactar a la madre antes de bloquearse preventivamente. Evita falsos positivos por caídas de la madre.

Lógica de gracia por fallo de endpoint

Hija intenta verificar → endpoint de la madre no responde.

→ ¿Días sin contacto < `grace_days_connection`? → sigue con aviso en panel.

→ ¿Días sin contacto >= `grace_days_connection`? → bloqueo preventivo.

Cuando se restaura la conexión y la madre confirma que está activo → desbloqueo automático.

6. Cálculo de expires_at (Regla Central)

6.1 La Regla

Fórmula

base = max(today, old_expires_at ?? today)

Desde base, saltar months_covered veces al próximo billing_day (con clamp).

El último billing_day alcanzado = nuevo expires_at.

6.2 Regla del billing_day en el Punto de Partida

Si base cae exactamente en el billing_day → ese día es el punto de partida, NO un salto consumido. El primer salto real siempre va al billing_day del mes siguiente.

6.3 Clamp al Último Día del Mes

Estándar de la industria (Stripe, PayPal). Si billing_day no existe en el mes destino → se usa el último día de ese mes.

billing_day	Mes	Día real usado	Razón
31	Febrero (normal)	28	Clamp al último día
31	Febrero (bisiesto)	29	Clamp al último día
31	Abril	30	Clamp al último día
31	Marzo	31	Existe, se usa normal
1-28	Cualquiera	billing_day	Siempre existe, sin clamp

6.4 Ejemplos Concretos

Ejemplo 1 — Primer pago (billing_day=7, grace=3)

Hoy: 7/4, old_expires_at: null, months_covered: 1

base = max(7/4, 7/4) = 7/4

Próximo billing_day desde 7/4 = 7/4 (punto de partida)

Salto 1 → 7/5

new_expires_at = 7/5

→ active: 7/4 al 7/5 | grace: 8/5 al 10/5 | suspended: 11/5+

Ejemplo 2 — Paga durante la gracia (billing_day=7, grace=3)

old_expires_at: 7/4 | Paga el 10/4 (día 3 de gracia) | months_covered: 1
base = $\max(10/4, 7/4) = 10/4$
Próximo billing_day desde 10/4 = 7/5
new_expires_at = 7/5
→ Los días 8-10/4 quedan dentro del período pagado. El corte siempre es el 7.

Ejemplo 3 — Pago anticipado de 3 meses (billing_day=7)

old_expires_at: 7/4 | Hoy: 1/4 | months_covered: 3
base = $\max(1/4, 7/4) = 7/4$ (no se castiga por pagar antes)
Salto 1 → 7/5 | Salto 2 → 7/6 | Salto 3 → 7/7
new_expires_at = 7/7

7. Sistema de Pagos

7.1 Tipos de Pago

Tipo	amount_cop	Descripción
normal	> 0	Pago regular mensual o de varios meses
promo	0	Mes de cortesía. Mueve expires_at igual que normal.
adjustment	> 0	Cobro puente por cambio de billing_day hacia adelante
credit	0 (a favor)	Saldo a favor por cambio de billing_day hacia atrás

7.2 Campos de la Tabla payments

Campo	Tipo	Descripción
id	UUID	Identificador único del pago
license_id	FK	Licencia a la que pertenece
amount_cop	integer	Monto en pesos colombianos
months_covered	integer	Meses que cubre este pago (mínimo 1)
type	enum	normal promo adjustment credit
notes	text	Notas opcionales del admin
paid_at	datetime	Fecha real del pago
recorded_by	FK user	Admin que registró el pago
consumed_amount_cop	integer	Solo para credits: cuánto se ha aplicado
consumed_at	datetime	Cuándo se agotó el crédito
voided_at	datetime	Si fue anulado (no se borra, se marca)
void_reason	text	Razón de la anulación

7.3 Regla de Inmutabilidad de Pagos

Qué se puede y qué no se puede editar

SE PUEDE EDITAR: amount_cop (si se digitó mal), notes, paid_at.

NO SE EDITA: Un pago confirmado no se borra. Se marca como anulado (voided_at) y se crea uno nuevo.

Esto garantiza trazabilidad total de todos los movimientos.

8. Sistema de Créditos

8.1 Cómo se Consumen los Créditos (FIFO)

Cuando hay créditos pendientes y se registra un pago normal, el sistema los aplica automáticamente en orden FIFO (el más antiguo primero).

Fórmula de aplicación

$\text{gross} = \text{price_cop} \times \text{months_covered}$

Recorrer créditos pendientes en orden (FIFO):

- Si crédito cubre todo el restante: marcar consumed_at, fin.
- Si crédito parcial: aplicar lo que alcanza, continuar con siguiente crédito.

$\text{net} = \max(\text{gross} - \text{créditos_aplicados}, 0)$

El admin ve el monto neto sugerido antes de confirmar el pago.

8.2 Crédito Parcial

Si un crédito de 50.000 COP se aplica contra un cobro donde solo se necesitan 30.000, el crédito queda con consumed_amount_cop = 30.000 y los 20.000 restantes siguen disponibles para el siguiente pago.

9. Cambio de billing_day

ADVERTENCIA

Cambiar el billing_day es una operación de alto riesgo.

Solo se permite en casos extremos.

El sistema muestra advertencia explícita y solicita confirmación antes de ejecutar.

El admin debe ver un preview del ajuste/crédito generado antes de confirmar.

9.1 Cambio Hacia Adelante (ej. día 7 → día 28)

Cálculo del cobro puente

$\text{precio_diario} = \text{ceil}(\text{price_cop} / \text{días_reales_del_mes_en_curso})$

$\text{días_extra} = \text{día_nuevo_real} - \text{día_viejo_real}$

$\text{cobro_puente} = \text{precio_diario} \times \text{días_extra}$

→ Se registra como payment type=adjustment

→ expires_at se realinea al nuevo billing_day (con clamp)

9.2 Cambio Hacia Atrás (ej. día 7 → día 4)

Cálculo del crédito a favor

$\text{precio_diario} = \text{ceil}(\text{price_cop} / \text{días_reales_del_mes_en_curso})$

$\text{días_crédito} = \text{día_viejo_real} - \text{día_nuevo_real}$

$\text{amount_credit} = \text{precio_diario} \times \text{días_crédito}$

→ Se registra como payment type=credit

→ expires_at se acorta al nuevo billing_day

→ El crédito se aplica automáticamente en el siguiente pago normal

9.3 Cambio de price_cop

Regla del cambio de precio

El precio nuevo aplica SOLO desde el siguiente ciclo.

El ciclo actual respeta el precio con el que fue pagado.

No se recalcula nada en el ciclo en curso.

10. Campos de la Tabla licenses

Campo	Tipo	Descripción
id	UUID	Identificador único de la licencia
client_id	FK	Cliente dueño de la licencia
license_key	string único	Clave secreta que usa la hija para autenticarse
plan_type	enum	monthly annual lifetime
billing_day	int 1-31	Día de corte mensual (con clamp). Null si lifetime.
price_cop	integer	Precio mensual en COP para esta licencia
grace_days	integer	Días de gracia post-vencimiento
expires_at	date	Fin del período pago (sin incluir gracia). Null si lifetime.
cancelled_at	datetime	Fecha de cancelación manual. Prioridad sobre todo.
created_at	datetime	Cuándo se creó la licencia
updated_at	datetime	Última modificación

11. Endpoint de Verificación

GET /api/license/verify

Autenticación

Cada hija envía su license_key única en el header.

La madre verifica que la key exista y pertenezca a un cliente real.

La respuesta va firmada con HMAC para que la hija pueda validar su autenticidad.

11.1 Request

Header: X-License-Key: {license_key}

Header: X-Signature: {hmac_firma}

Header: X-Timestamp: {timestamp}

11.2 Response

Campos de la respuesta

status: active | grace_period | suspended | cancelled | trial

plan_type: monthly | annual | lifetime

expires_at: fecha de vencimiento (null si lifetime)

billing_day: día de corte (null si lifetime)

price_cop: precio mensual actual

grace_days: días de gracia configurados

in_grace: boolean — si está actualmente en período de gracia

next_cutoff_at: fecha del próximo corte calculado

checked_at: timestamp de la consulta (para que la hija lo guarde)

12. Módulo license-guard (Hija)

12.1 Qué es

Paquete JS reutilizable que se instala en cualquier proyecto hija. Se configura con pocas líneas y expone un middleware Express listo para usar.

12.2 Estado Local que Guarda la Hija

Campo	Descripción
lastKnownStatus	Último estado que respondió la madre
lastCheckedAt	Timestamp de la última consulta exitosa
planType	Para saber cada cuánto debe verificar
expiresAt	Copia local de la fecha de vencimiento
graceDaysUsed	Días consecutivos sin poder contactar a la madre

12.3 Frecuencia de Verificación por Plan

Plan	Frecuencia	Razón
monthly	Cada 3 días	Ciclo corto, detección rápida
annual	Cada 7 días	Más margen
lifetime	Cada 30 días	Solo detecta revocaciones manuales

12.4 Flujo Completo de Decisión

Al hacer login el admin

1. ¿Pasaron N días desde lastCheckedAt?

NO → usar lastKnownStatus cacheado → aplicar middleware según estado

SÍ → consultar endpoint de la madre

2. Si la madre responde:

active → guardar estado + resetear graceDaysUsed → continuar normal

grace → guardar estado → continuar con aviso en panel

suspended → guardar estado → activar bloqueo total

3. Si la madre NO responde (timeout/error):

`graceDaysUsed < grace_days_connection` → continuar con aviso en panel

`graceDaysUsed >= grace_days_connection` → bloqueo preventivo

13. Dashboard de la Página Madre

13.1 Vista por Licencia

- billing_day, price_cop, status actual, próximo corte calculado
- expires_at y días restantes
- Indicador visual de estado (active / grace / suspended / cancelled)
- Créditos pendientes disponibles

13.2 Historial de Pagos

- Todos los movimientos: normal, promo, adjustment, credit, void
- Fecha, monto, meses cubiertos, quién lo registró, notas
- Pagos anulados visibles pero marcados (nunca se borran)

13.3 Formulario de Registro de Pago

- Selector de tipo: normal / promo
- Monto en COP (0 si promo)
- Meses cubiertos
- Muestra créditos pendientes y el monto neto sugerido automáticamente
- Preview del nuevo expires_at antes de confirmar
- Campo de notas opcionales

13.4 Formulario de Cambio de billing_day

- Input del nuevo día (1-31)
- Preview automático del adjustment o credit que se generará
- Advertencia explícita de que es una operación de riesgo
- Confirmación requerida antes de ejecutar

13.5 Cambio de price_cop

- Input del nuevo precio
- Aviso claro: aplica solo desde el siguiente ciclo
- No afecta el ciclo actual

13.6 Vista de Clientes Vencidos

Cuando el admin de la madre entra al panel, ve automáticamente la lista de clientes que vencieron o están en grace desde la última vez que revisó. No requiere cronjob: se calcula al cargar la vista.

14. Seguridad del Endpoint

Mecanismos de seguridad

- license_key única por hija: generada al registrar el cliente, nunca compartida.
- Firma HMAC en cada request: la hija firma con su secret, la madre valida.
- Firma HMAC en cada response: la madre firma, la hija valida antes de aceptar.
- Timestamp en el request: evita replay attacks (ventana de ± 5 minutos).
- Sin la key y la firma correctas, el endpoint no responde información real.

15. Resumen de Todas las Reglas de Negocio

Reglas de expires_at

- $base = \max(today, old_expires_at \text{ ?? } today)$
- Saltar months_covered veces al próximo billing_day con clamp al último día del mes
- Si base cae en billing_day → es punto de partida, no salto consumido

Reglas de Estado

- El estado se calcula en tiempo real, no se almacena como campo fijo
- cancelled_at tiene prioridad sobre cualquier otro estado
- lifetime solo puede ser suspendido manualmente vía cancelled_at

Reglas de Pagos

- Los pagos confirmados no se borran, solo se anulan (voided_at)
- promo se comporta igual que normal, solo el amount es 0
- Los créditos se aplican FIFO automáticamente al registrar un pago normal
- price_cop cambia solo para el siguiente ciclo, nunca el actual

Reglas de billing_day

- Solo se cambia en casos extremos, con advertencia y preview
- Cambio adelante → adjustment (cobro puente)
- Cambio atrás → credit (saldo a favor)
- $precio_diario = \text{ceil}(price_cop / \text{días_reales_del_mes_en_curso})$

Reglas de Bloqueo

- El bloqueo del cliente opera a nivel de middleware, independiente del admin
- La hija bloquea preventivamente si no puede contactar a la madre por X días
- Al cancelar, se respeta el expires_at actual antes de bloquear